

Analysis Report — Cursor vs GitHub Copilot

Lab 2 — Phân tích sản phẩm AI | Ngành [B] Lập trình Nhóm:
Quách Ngọc Quang (2A202600285) · Nguyễn Đông Hưng
(2A202600392)

Cover

Yếu tố	Thông tin
Ngành	[B] Lập trình
Sản phẩm A	Cursor — https://www.cursor.com
Sản phẩm B	GitHub Copilot — https://github.com/features/copilot
Nhiệm vụ chung	Viết hàm Python tính khoảng cách Levenshtein giữa 2 chuỗi, có kèm unit test và giải thích chi tiết phần xử lý edge cases
Thành viên	2A202600285 — Quách Ngọc Quang · 2A202600392 — Nguyễn Đông Hưng

S1 — Product Moment

1.1 Bảng so sánh Entry Point

Yếu tố	Cursor	GitHub Copilot
URL	cursor.com	github.com/features/copilot
Entry point	Chat panel trong IDE (phím tắt <code>Ctrl + L</code>)	Copilot Chat trong VS Code / GitHub web
Ý định user	Pair programming agent hiểu toàn bộ codebase	AI assistant gợi ý code inline + chat
Surface chính	IDE fork từ VS Code với AI-native UI	VS Code extension + GitHub web interface

1.2 Nhiệm vụ chung

Prompt đã dùng:

"Viết hàm Python tính khoảng cách Levenshtein giữa 2 chuỗi, có kèm unit test và giải thích chi tiết phần xử lý edge cases."

1.3 Nhận định

Cursor thắng ở **Product Moment**: AI được thiết kế làm trung tâm của editor ngay từ đầu — người dùng gõ prompt và nhận code ngay trong IDE, không cần chuyển context.

GitHub Copilot thắng ở **Distribution**: nằm sẵn trong hệ sinh thái GitHub/VS Code mà developer đã quen dùng, không cần cài thêm tool mới hay thay đổi workflow.

S2 — Workflow Evidence

2.1 Luồng người dùng

Giai đoạn	Cursor	GitHub Copilot
TRƯỚC khi có AI	Open browser → Stack Overflow → Copy code → Paste vào IDE → Tự viết unit test → Debug (10–15 phút)	Quy trình tương tự: tìm tài liệu thủ công, viết code và test từ đầu
TRONG (có AI)	[1] Gõ prompt vào Chat panel → [2] AI sinh toàn bộ hàm + unit test + giải thích → [3] Nhấn "Apply" để chèn thẳng vào file → [4] Chạy test trong terminal tích hợp	[1] Mở Copilot Chat (VS Code) → [2] AI sinh code và test → [3] Chấp nhận inline suggestion hoặc copy thủ công → [4] Chạy test trong terminal riêng
SAU khi có AI	Code chạy được sau khi fix một vài minor issues; tiết kiệm ~10 phút so với trước	Code chạy được sau khi điều chỉnh nhỏ; trải nghiệm tương đương nhưng bước chèn code tốn thêm thao tác

2.2 Ba Friction Areas

Friction Type	Cursor	GitHub Copilot
Physical load (click, copy-paste)	Thấp — mọi thứ trong 1 IDE, nút "Apply" chèn code không cần copy	Trung bình — inline suggestion tiện, nhưng Copilot Chat yêu cầu copy/paste thủ công
Cognitive burden (học prompt?)	Thấp — chat bằng ngôn ngữ tự nhiên, không cần biết cú pháp đặc biệt	Thấp — tương tự, giao tiếp tự nhiên
User workarounds	Phải gõ lệnh chạy test thủ công trong terminal	Phải setup extension + cấu hình context repo nếu lần

Friction Type	Cursor	GitHub Copilot
	(chưa auto-run)	đầu dòng

2.3 Nhận định

Cursor giảm friction tốt hơn nhờ **AI-native workflow**: toàn bộ vòng lặp prompt → sinh code → apply diễn ra trong cùng 1 màn hình. GitHub Copilot vẫn hiệu quả nhưng bị phân tán khi nằm ở nhiều surface (VS Code extension, GitHub web, CLI) — mỗi surface lại có context riêng.

S3 — Output & Trust

3.1 Chất lượng output

Tiêu chí	Cursor	GitHub Copilot
Đúng nội dung?	Có — hàm Levenshtein chuẩn, test case bao phủ đủ trường hợp	Có — output tương đương, logic chính xác
Có bịa không?	Không — code chạy được ngay, không có hallucination	Không — code chính xác, không bịa API hay method
Tiếng Việt tự nhiên?	Rất tự nhiên — giải thích 8 edge cases đầy đủ bằng tiếng Việt	Phần lớn tiếng Anh; tiếng Việt có nhưng ngắn hơn
Đầy đủ các phần?	Đầy đủ: hàm chính + unit test + giải thích edge cases	Đầy đủ: hàm chính + unit test; giải thích edge cases ngắn hơn
Tốc độ trả lời	~5–10 giây	~5–15 giây

3.2 Sáu tín hiệu đáng tin (Trust Signals)

Tín hiệu	Cursor	GitHub Copilot
Citation	Không có (code generation không cần citation)	Không có
Control (sửa/làm lại)	Có — nút "Apply", "Retry", edit trực tiếp trong IDE	Có — chấp nhận/từ chối inline; Copilot Chat có regenerate
Confidence indicator	Không hiển thị	Không hiển thị
Failure handling	Giải thích lỗi + đề xuất fix cụ thể khi test fail	Tương tự — gợi ý fix khi có lỗi syntax hoặc logic
Feedback (báo lỗi)	Chưa có tính năng báo lỗi tích hợp trực tiếp	Chưa có — phụ thuộc GitHub Issues/Feedback portal
Handoff to human	Unit test để tự chứng minh correctness	Unit test + PR review workflow trên GitHub

3.3 Nhận định

Với code generation, **trust không đến từ citation** mà đến từ khả năng **execute và verify bằng unit test**. Cursor tạo cảm giác tin nhanh hơn vì output chạy được ngay trong IDE. GitHub Copilot có trust bền hơn ở môi trường team nhờ tích hợp sâu vào GitHub PR/review workflow.

S4 — Business / Usage Signal

4.1 Mô hình giá

Yếu tố	Cursor	GitHub Copilot
Gói miễn phí	Hobby — giới hạn model và slow requests	Copilot Free — giới hạn completions/chat
Giá thấp nhất	\$20/tháng (Pro)	\$10/tháng (Copilot Pro)
Mô hình tính phí	Subscription — unlimited requests trong quota	Subscription — unlimited trong plan
Quota / limit	500 fast requests/tháng (Pro)	Giới hạn tùy plan
Khi hết quota	Tự động chuyển sang slow requests; hiện thông báo upgrade	Hiện thông báo upgrade
Tiers	Free / Pro (\$20) / Pro+ / Ultra	Free / Pro (\$10) / Business (\$19/user) / Enterprise (\$39/user)

4.2 Cost–Capability–Speed

Sản phẩm	Cost	Capability	Speed	Trade-off chính
Cursor	\$20/tháng	Context toàn codebase, multi-file edit, agent mode	Rất nhanh — tiết kiệm ước tính 2–3x	Đắt hơn 2x Copilot, nhưng capability và UX cao hơn rõ rệt
GitHub Copilot	\$10/tháng	Inline suggestions, Copilot Chat, repo context	Nhanh	Rẻ hơn, phù hợp enterprise; UX ít liền mạch hơn

4.3 Nhận định

Cursor có **pricing power** mạnh ở phân khúc cá nhân: developer sẵn sàng tự bỏ \$20/tháng vì giá trị mang lại rõ ràng. GitHub Copilot có **distribution moat** và **enterprise advantage** nhờ bundling với GitHub/Microsoft — dễ được phê duyệt ngân sách công ty hơn.

S5 — Product Judgment

S5.1 Verdict

Sản phẩm	Verdict	Lý do
Cursor	STRONG	AI-native IDE với product moment rõ ràng; workflow mượt mà, willingness-to-pay cao ở individual developer
GitHub Copilot	STRONG	Distribution và ecosystem mạnh; switching cost cao; phù hợp team/enterprise hơn Cursor

S5.2 User Base + Tăng trưởng

Chỉ số	Cursor	GitHub Copilot	Nguồn
Paying users	~720,000 (ước tính)	20+ triệu users	Sacra; Microsoft AR 2025
ARR	~\$200M (ước tính)	Không tách rõ	Sacra
Growth rate	Hypergrowth 2024–2025	Steady enterprise adoption	Sacra; Microsoft IR

Nhận định: Cursor tăng trưởng theo mô hình PLG (product-led growth) — cá nhân dùng trước, trả tiền sau. Copilot tăng đều nhờ enterprise bundling và GitHub ecosystem.

S5.3 Doanh thu / Pricing Power

Chỉ số	Cursor	GitHub Copilot	Nguồn
ARR	~\$200M (ước tính)	Không tách rõ khỏi Microsoft	Sacra
Giá thấp nhất	\$20/tháng	\$10/tháng	Official pricing
Pricing strategy	Freemium → Premium cá nhân	Freemium → Enterprise seat	
Tiers	Free / Pro / Pro+ / Ultra	Free / Pro / Business / Enterprise	Official

Nhận định: Cursor pricing power mạnh ở power users cá nhân — ARPU cao hơn. Copilot mạnh hơn ở enterprise seat expansion — volume lớn, margin ổn định.

S5.4 Moat Phân tích (5 loại)

Loại Moat	Cursor	GitHub Copilot
Data moat	Trung bình — interaction trong workspace người dùng	Cao — toàn bộ GitHub repo, PR, issue, code review
Network effect	Thấp–trung bình	Cao — GitHub developer ecosystem

Loại Moat	Cursor	GitHub Copilot
Switching cost	Trung bình — import settings từ VS Code dễ	Cao — workflow team gắn chặt vào GitHub/CI
Brand	Mạnh trong cộng đồng AI-native coding	Rất mạnh — GitHub + Microsoft
Distribution	Trung bình — PLG, organic	Rất mạnh — GitHub/VS Code/enterprise bundling

Moat chủ đạo: - **Cursor:** Product moat (UX + agent workflow) — có thể bị copy nếu model bị commoditize - **Copilot:** Distribution moat — khó phá hơn, không phụ thuộc vào 1 model duy nhất

S5.5 Data Flywheel + Feedback Loop

Câu hỏi	Cursor	GitHub Copilot
Hành động feed model	Accept/reject edits, workspace context, custom rules	Completions accepted, PR diffs, code review comments, repo context
Loop có compounding?	Có một phần — workflow cá nhân hóa theo thời gian	Có mạnh hơn — ecosystem rộng, data đa dạng từ triệu repo
Feedback systematic?	Usage dashboard, rules/hooks, <code>.cursorrules</code>	Telemetry ẩn, enterprise có privacy controls riêng
Rủi ro big tech	Nếu model nền bị commoditize, Cursor phải giữ UX loop	Copilot phụ thuộc Microsoft AI stack — ít tự chủ về model

Nhận định: Flywheel bền nhất không phải "model thông minh hơn" mà là **context + workflow + habit** — ai lock-in được 3 thứ này thắng dài hạn.

S5.6 Niche Down + AI Feature Map

Chiều	Cursor	GitHub Copilot
Niche rõ?	Có — AI-native IDE cho individual developer muốn tốc độ cao	Có — AI pair programmer tích hợp vào GitHub/VS Code workflow tổ chức
User Value	Rất cao với cá nhân / power user	Cao với developer làm việc trong repo thật
User Alignment	Cao với individual dev; thấp hơn khi team cần governance	Cao với team dùng GitHub; tích hợp tự nhiên vào review process
Business Value	Cao — willingness-to-pay \$20+ rõ ràng	Rất cao — enterprise seat expansion, dễ phê duyệt ngân sách

S5.7 Spark → Loop → System

Giai đoạn	Cursor	GitHub Copilot
Spark	Wow đầu tiên khi gõ prompt → AI sinh code + test → nhấn Apply xong ngay	Inline suggestion xuất hiện tự động; Copilot Chat trả lời ngay trong editor
Loop	Developer quay lại vì editor đã trở thành pair programmer — không muốn code không có AI	Quay lại vì AI gắn vào repo/PR/issue — AI xuất hiện tự nhiên trong mọi bước workflow

Giai đoạn	Cursor	GitHub Copilot
System	Đang xây dựng — Cursor Rules, MCP server, cloud agents (chưa hoàn chỉnh)	Gần system hơn — GitHub/Microsoft ecosystem, Copilot Workspace, CLI, mobile

Dự báo 12 tháng: - **Cursor** cần chứng minh là "coding OS" thực sự, không chỉ là wrapper thông minh quanh model. Nếu không giữ được UX moat, dễ bị commoditize. - **GitHub Copilot** cần tránh cảm giác intrusive khi ép AI vào mọi nơi; rủi ro là developer dùng vì bắt buộc, không phải vì yêu thích.

S5.8 Liên hệ Lab 1 Case (Fiverr)

Bài học từ Fiverr case áp dụng cho Lab 2:

- Workflow quyết định thắng, không phải output:** Cursor thắng vì ở ngay trong editor; Copilot thắng vì ở ngay trong GitHub workflow — tương tự Fiverr, sản phẩm nào nằm sẵn trong flow của user sẽ giữ được user lâu hơn, dù output có tương đương.
- Moat phải nằm trong habit và context:** Fiverr mất buyer khi chat interface thay thế search; Copilot có lợi thế tương tự vì nằm trong repo workflow — user không cần chuyển context, không có lý do để switch.
- Task đơn giản sẽ bị commoditize:** Autocomplete cơ bản đã là commodity — cả Cursor và Copilot đều phải đi lên agentic workflow, repo-scale context và team governance để giữ pricing power.

Sản phẩm	Rủi ro disruption
Cursor	Rủi ro cao hơn nếu model nền bị commoditize và đối thủ copy được UX moat — moat hiện tại mỏng hơn Copilot
GitHub Copilot	Rủi ro thấp hơn nhờ distribution moat + enterprise lock-in; rủi ro chính là bị cảm nhận là "ép AI" thay vì "AI hữu ích"

Nguồn tham khảo

1. Cursor Pricing — <https://cursor.com/pricing>
2. Sacra, Cursor at \$200M ARR — <https://sacra.com/research/cursor-at-200m-arr/>
3. GitHub Copilot Plans — <https://github.com/features/copilot/plans>
4. Microsoft Annual Report 2025 — <https://www.microsoft.com/investor/reports/ar25/index.html>
5. Lab 1 FINAL case Fiverr — `../01-bigtech-disruption/3-FINAL-case-analysis.md`